❶ Configures your HSTS header settings and changes the MaxAge from the default of 30 days

❷ You shouldn’t use HSTS in local environments.

❸ Adds the HstsMiddleware

The preceding example shows how to change the `MaxAge` sent in the HSTS header. It’s a good idea to start with a small value initially. Once you’re sure your app’s HTTPS is functioning correctly, you can increase the age for greater security. A typical value for production deployments is one year.

WARNING Once client browsers have received the HSTS header, browsers will default to using HTTPS for all requests to your application. That means you must commit to always using HTTPS for as long as you set `MaxAge`. If you disable HTTPS, browsers will not revert to using HTTP until this duration has expired, so your application may be inaccessible until then if you aren’t listening on HTTPS! You can notify the browser that your app no longer supports HSTS by setting `MaxAge` to `0`.

One limitation with the HSTS header is that you must make an initial request over HTTPS before you can receive the header. If the browser makes only HTTP requests, the app never has a chance to send the HSTS header, so the browser never knows to use HTTPS. One potential solution is called HSTS *preload*.

HSTS preload isn’t part of the HSTS specification, but it’s supported by all modern browsers. Preload bakes your HSTS header into the browser so that the browser knows it should make only HTTPS requests to your site. That removes the “first request” problem entirely, but be aware that HSTS preload commits you to HTTPS forever, as it can’t easily be undone.

Once you’re comfortable with your application’s HTTPS configuration, you can prepare your app for HSTS preload by configuring an HSTS header that